

算法竞赛核心知识库与刷题心法

一、 数据结构

1. 树状数组 (BIT)

树状数组是一种基于二进制思想的数据结构，其基本用途是**维护序列的前缀和**。

对于给定的序列 a ，我们建立一个数组 c ，其中 $c[x]$ 保存序列 a 的区间 $[x - \text{lowbit}(x) + 1, x]$ 中所有数的和。

前置知识：lowbit 定义

`lowbit(n)` 定义为非负整数 n 在二进制表示下“最低位的 1 及其后边所有的 0”。

```
1 // lowbit 求法
2 int lowbit(int x) { return x & (-x); }
```

树状数组核心模板：

```
1 struct BIT {
2     int n;
3     vector<int> tree;
4     BIT(int n) : n(n), tree(n + 1, 0) {}
5
6     int lowbit(int x) { return x & -x; }
7
8     // 单点增加
```

```

9     void add(int x, int d) {
10         for (int i = x; i <= n; i += lowbit(i)) tree[i] += d;
11     }
12
13     // 查询前缀和 [1, x]
14     int ask(int x) {
15         int sum = 0;
16         for (int i = x; i > 0; i -= lowbit(i)) sum += tree[i];
17         return sum;
18     }
19 };

```

2. 线段树

支持区间修改以及区间查询的高阶数据结构

听完线段树1的讲解后本人认为线段树的Node构造需要遵循

首先l与r是必须的，其次就是答案让查询的东西，然后我们思考这个东西是否能被递归pushup更新上去，如果不能我们就继续把这个元素拆分成可以递归更新的属性，这便是框架

听完线段树2的讲解后本人心得

好他妈麻烦啊啊啊啊啊，首先懒标记太重要，其次如果有多个懒标记需要有一个好用的顺序，方便你写代码修改pushdown和pushup操作

大致模板如下：

```

1 struct SegTree {
2     //本模板对应题意，区间修改，区间求和
3     struct Node {

```

```

4     int l, r;
5     int sum, add;
6 } tr[4 * N];
7
8 vector<int> w;
9
10 void Sread(int n) {
11     w.assign(n + 1, 0);
12     for(i, 1, n) cin>>w[i];
13 }
14
15 void pushup(Node &u, Node &l, Node &r) { //一份pushup对自己
16     u.sum = l.sum + r.sum;
17 }
18
19 void pushup(int u) { //对外
20     pushup(tr[u], tr[u << 1], tr[u << 1 | 1]);
21 }
22
23 void pushdown(Node &u, Node &l, Node &r) { //pushdown懒标记的更新
24     l.sum += (l.r - l.l + 1) * u.add;
25     r.sum += (r.r - r.l + 1) * u.add;
26
27     l.add += u.add;
28     r.add += u.add;
29
30     u.add = 0;
31 }
32
33 void pushdown(int u) {
34     pushdown(tr[u], tr[u << 1], tr[u << 1 | 1]);
35 }
36
37 void build(int u, int l, int r) { //建树
38     if (l == r) tr[u] = {l, r, w[l], 0};
39     else {
40         tr[u].l = l, tr[u].r = r;
41         int mid = (l + r) >> 1;
42         build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);
43         pushup(u);
44     }
45 }
46

```

```

47 void modify(int u, int l, int r, int d) { //修改
48     if (l <= tr[u].l && tr[u].r <= r) {
49         tr[u].sum += (tr[u].r - tr[u].l + 1) * d;
50         tr[u].add += d;
51     }
52     else {
53         pushdown(u);
54         int mid = (tr[u].l + tr[u].r) >> 1;
55         if (l <= mid) modify(u << 1, l, r, d);
56         if (r > mid) modify(u << 1 | 1, l, r, d);
57         pushup(u);
58     }
59 }
60
61
62 Node query(int u, int l, int r) { //查询
63     if (l <= tr[u].l && tr[u].r <= r) return tr[u];
64
65     pushdown(u);
66     int mid = (tr[u].l + tr[u].r) >> 1;
67     if (r <= mid) return query(u << 1, l, r);
68     if (l > mid) return query(u << 1 | 1, l, r);
69     else {
70         auto left = query(u << 1, l, r);
71         auto right = query(u << 1 | 1, l, r);
72         Node res;
73         pushup(res, left, right);
74         return res;
75     }
76 }
77 };

```

3. 链表模板

```

1 struct List {
2     int e[N], l[N], r[N];
3     int idx;
4 }

```

```
5 List () {
6     init();
7 }
8
9 void init() {
10     idx = 2;
11     r[0] = 1;
12     l[1] = 0;
13 }
14
15 void insert_right(int k, int x) {
16
17     e[idx] = x;
18     l[r[k]] = idx;
19     r[idx] = r[k];
20     l[idx] = k;
21     r[k] = idx;
22     idx++;
23 }
24
25 void add_head(int x) {
26     insert_right(0, x);
27 }
28
29 void add_tail(int x) {
30     insert_right(l[1],x);
31 }
32
33 void remove(int k) {
34     r[l[k]] = r[k];
35     l[r[k]] = l[k];
36 }
37
38 void insert_left(int k, int x) {
39     insert_right(l[k], x);
40 }
41
42
43 };
44
```

4. 队列模板

```
1 int queues[N];
2 int tt=-1,hh;
3 void push(int x)
4 {
5     queues[++tt]=x;
6 }
7 void pop()
8 {
9     hh++;
10 }
11 void query()
12 {
13     cout<<queues[hh]<<"\n";
14 }
15 void empty()
16 {
17     if(tt<hh)cout<<"YES"<<"\n";
18     else cout<<"NO"<<"\n";
19 }
```

5. 栈模板

```
1 int stk[N],tt = 0;
2
3 void add(int x)
4 {
5     stk[++tt] = x;
6 }
7 void pop()
8 {
9     tt--;
10 }
11 int top()
12 {
```

```
13     return stk[tt];
14 }
15 bool empty()
16 {
17     return tt==0;
18 }
19
```

二、算法思想与策略

1. 二维偏序（扫描线降维打击）

核心本质：利用“排序”消灭一个维度，再用数据结构（BIT）动态维护另一个维度。

通俗理解（HR 招聘模型）：

假设求职者有两个属性：**笔试成绩** (X) 和 **面试成绩** (Y)。要求找出 $X \leq 80$ 且 $Y \leq 90$ 的人。

直接嵌套循环会导致 $O(N^2)$ 。我们的降维魔法如下：

- 1. 消灭第一维 X ：**将所有人按笔试成绩 X 从低到高排队。现在我们从左到右挨个审核，审核到第 k 个人时，前面的 $k-1$ 个人 X 必然满足条件！这样 X 维就被**排序动作**消灭了。
- 2. 维护第二维 Y ：**准备一个一维的树状数组（黑板）专门记录面试成绩 Y 。每审核过一个人就把他的 Y 登记上去。查询时，直接在这个一维黑板上求 Y 的前缀和即可。

2. 离散化

当坐标值域极大（如 10^9 ）但元素个数有限（如 10^5 ）时，将数值映射为 $1 \sim N$ 的紧凑排名，以便存入树状数组。

离散化黄金三连：复制 -> 排序+去重 -> 二分找排名

```

1 vector<int> b = a;
2 sort(b.begin(), b.end());
3 b.erase(unique(b.begin(), b.end()), b.end());
4
5 for (int i = 0; i < n; i++) {
6     // lower_bound 返回迭代器，减去 begin() 得到 0-based 下标。加 1 变成 BIT 1-based
    排名
7     a[i] = lower_bound(b.begin(), b.end(), a[i]) - b.begin() + 1;
8 }

```

3. 归并排序求逆序对

注意右半段较小时才会产生逆序对。

```

1 int a[N], tmp[N];
2
3 int merge_sort(int l, int r) {
4     if (l >= r) return 0;
5
6     int mid = l + (r - l) / 2; // 【防爆排雷】：不要写 l + r >> 1，注意优先级和防溢
    出
7     int res = merge_sort(l, mid) + merge_sort(mid + 1, r);
8
9     int i = l, j = mid + 1, k = 0;
10    while (i <= mid && j <= r) {
11        if (a[i] <= a[j]) {
12            tmp[k++] = a[i++];
13        } else {
14            // 右边较小，构成逆序对！
15            // 因为左半段 [i, mid] 是有序的，a[i] 到 a[mid] 都大于 a[j]
16            res += mid - i + 1;
17            tmp[k++] = a[j++];
18        }
19    }
20
21    while (i <= mid) tmp[k++] = a[i++];

```



```

22     while (j <= r) tmp[k++] = a[j++];
23     for (i = 1, k = 0; i <= r; i++, k++) a[i] = tmp[k];
24
25     return res;
26 }

```

4. 懒惰更新与时间戳

面对高频的全局覆盖（`a = d`）操作，绝对不写 $O(n)$ 的硬拷贝。

- **策略：**维护一个全局 `last_reset` 和单点的 `last_change[i]`。
- **致命陷阱：**在对任何数据进行判断之前，必须先校验它的时间戳！如果过期了，必须先“唤醒”恢复真实值，用假数据做判断是 Lazy Update 的大忌。

5. 数学公式的合理拆分

当你推理出来本题目的数学公式的时候，应当往下一直化简。

- **思想：**分离包含有多个变量的项，使公式中不同变量之间互相独立

例子：

$$\sum_{i=1}^x \sum_{j=1}^i b_j = \sum_{i=1}^x (x - i + 1) * b_i$$

$$\sum_{i=1}^x (x - i + 1) * b_i = (x + 1) \sum_{i=1}^x b_i - \sum_{i=1}^x i * b_i$$

这样一来，该数学式子就可以拆分成两个前缀和，都可以实现

6.快速排序板子

```
1  auto qsort = [&](auto&&self,int l,int r)->void{
2      if(l>=r) return;
3      int left = l-1;
4      int right = r + 1;
5      int base = a[(left+right)>>1];
6
7      while(left<right){
8          while(a[++left]<base);
9          while(a[--right]>base);
10         if(left<right) swap(a[left],a[right]);
11     }
12 }
```

三、图论与搜索

1. 拆环成链

核心思想：如何简洁地表示环中两点距离？开一个二倍数组，将环拆成链子。

`pre[n+i]` 的含义是从起点出发走两圈到该点。例如：顺时针 6 点到 4 点的距离 = 走两圈到 4 点的距离 - 走一圈到 6 点的距离。

```
1  // 核心逻辑：s 为第一圈的总周长
2  for (int i = 1; i <= n; i++) {
3      pre[i + n] = pre[i] + s;
4  }
```

2. DFS / 回溯法

DFS 虽然千变万化，但万变不离其宗。核心在于**保存现场与恢复现场**。

```
1 void dfs(int step) {
2     if (/* 到达终点 或 满足边界条件 */) {
3         // 记录答案或输出
4         return;
5     }
6
7     for (/* 遍历当前状态的所有可能选择 */) {
8         if (/* 该选择合法且未被访问 */) {
9             // 1. 标记该选择已被访问
10            // 2. 保存现场 (应用该选择)
11
12            dfs(step + 1); // 递归进入下一层
13
14            // 3. 恢复现场 (撤销该选择) — 回溯的核心!
15            // 4. 解除访问标记
16        }
17    }
18 }
```

四、数学与数论

1. 取模极限规律

核心定理：当 $B < A$ 时，对于给定的整数 A 对任意正整数 B 取模，其最大可能取值为：

$$A \bmod B \leq \lfloor \frac{A-1}{2} \rfloor$$

(注：当且仅当 $B = \lfloor \frac{A}{2} \rfloor + 1$ 时取到最大值。如果 $B > A$ ，则 $A \bmod B = A$)。

2. MEX

指在一个非负整数集合中**不包含的最小非负整数**。

- 例：集合 $S = \{0, 1, 2, 3\}$ ，那么 $\text{mex}(S) = 4$ 。

3. 连续整数异或和的周期性质

定义前缀异或和 $P(i) = 1 \oplus 2 \oplus 3 \oplus \dots \oplus i$ ，规定 $P(0) = 0$ 。

$P(i)$ 的取值具有以 4 为周期的强规律：

- $P(i) = i$ ，若 $i \equiv 0 \pmod{4}$
- $P(i) = 1$ ，若 $i \equiv 1 \pmod{4}$
- $P(i) = i + 1$ ，若 $i \equiv 2 \pmod{4}$
- $P(i) = 0$ ，若 $i \equiv 3 \pmod{4}$

4. 哥德巴赫猜想

哥德巴赫猜想的现代陈述为：**任一大于5的整数都可写成三个质数之和**。

由此可以得到两种情况：

- 1.强哥德巴赫猜想（强哥猜）：**即任一大于2的偶数都可写成两个质数之和**；（未被证明，但是同时也没有被推翻，即在本体的范围内强哥猜成立）
- 2.弱哥德巴赫猜想（弱哥猜）：**任何一个大于5的奇数都能被表示成三个奇质数的和**。（一个质数可以被多次使用）；（已经被证明）

五、 考场通关与刷题心法

- ☐ **翻译题目**：强迫自己用一句话、严谨的数学语言重新描述题意，剥离冗余背景。

- **极限施压**：要想出能卡掉程序的**极限数据与边界**，穷举所有极端情况，防止漏判。
- **拒绝干瞪眼**：遇到抽象限制，必须**纸笔手工模拟**，自己去造数据理解流程。
- **复杂度反推**：从数据范围反推时间复杂度（例如看到 10^5 盲猜 $O(N \log N)$ ），再去对应层级的算法和数据结构里找武器。
- **避坑指南**：字符串哈希容易冲突且极难调试，能用 `std::map` 转化为数字 ID 的，就优先转成数字 ID！
- **查询越界思维**：用 `upper_bound - begin()` 求 $\leq target$ 的个数时，算出来的本质是“越界者的排名”，直接用 `ask(rank - 1)` 查询，逻辑闭环最严密。

六、 矩阵与坐标技巧

主副对角线恒等式：

- 主对角线（左上-右下）： $i - j = \text{const}$ 。
- 副对角线（右上-左下）： $i + j = \text{const}$ 。

坐标转换应用：在 `X-Sum` 题中，利用此规律可将 $O(N^2)$ 的动态搜索优化为 $O(1)$ 的预处理读取。

附录：实战长代码留档

【实战例题：配对统计】完整扫描线代码

```
1 #include <bits/stdc++.h>
2 #define int long long
3 using namespace std;
4 const int N = 3e5+10;
```

```

5
6 int test, n, m;
7 int f[N];
8
9 void add(int x, int y) {
10     for (; x <= n; x += x & (-x)) f[x] += y;
11 }
12
13 int ask(int x) {
14     int res = 0;
15     for (; x; x -= x & (-x)) res += f[x];
16     return res;
17 }
18
19 void solve() {
20     cin >> n >> m;
21     if (n == 1) {
22         cout << "0\n";
23         return;
24     }
25     vector<pair<int, int>> a(n + 1);
26     for (int i = 1; i <= n; i++) {
27         cin >> a[i].first;
28         a[i].second = i;
29     }
30
31     sort(a.begin() + 1, a.end(), [](const auto & aa, const auto & bb) {
32         return aa.first < bb.first;
33     });
34
35     vector<pair<int, int>> pairs;
36
37     // 处理首尾情况
38     int l = min(a[1].second, a[2].second), r = max(a[1].second, a[2].second);
39     pairs.push_back({l, r});
40     l = min(a[n].second, a[n - 1].second), r = max(a[n].second, a[n -
41 1].second);
42     pairs.push_back({l, r});
43
44     for (int i = 2; i < n; i++) {
45         int ldif = a[i].first - a[i - 1].first;
46         int rdif = a[i + 1].first - a[i].first;
47         if (ldif < rdif) {

```

```

47         int x = a[i].second, y = a[i - 1].second;
48         if (x > y) swap(x, y);
49         pairs.push_back({x, y});
50     } else if (ldif == rdif) {
51         int x1 = a[i].second, y1 = a[i - 1].second;
52         int x2 = a[i].second;
53         int y2 = a[i + 1].second;
54         if (x1 > y1) swap(x1, y1);
55         if (x2 > y2) swap(x2, y2);
56         pairs.push_back({x1, y1});
57         pairs.push_back({x2, y2});
58     } else {
59         int x = a[i].second, y = a[i + 1].second;
60         if (x > y) swap(x, y);
61         pairs.push_back({x, y});
62     }
63 }
64
65 sort(pairs.begin(), pairs.end(), [](const auto & aa, const auto & bb) {
66     if (aa.second != bb.second) return aa.second < bb.second;
67     return aa.first < bb.first;
68 });
69
70 int ans = 0, pair_idx = 0;
71 int total_pairs = pairs.size();
72
73 vector<pair<int, pair<int, int> > > dicar;
74 for (int i = 1; i <= m; i++) {
75     int l, r;
76     cin >> l >> r;
77     dicar.push_back({i, {l, r}});
78 }
79
80 sort(dicar.begin(), dicar.end(), [](const auto & aa, const auto & bb) {
81     return aa.second.second < bb.second.second;
82 });
83
84 // 扫描线核心逻辑
85 for (auto &it : dicar) {
86     while (pair_idx < total_pairs && pairs[pair_idx].second <=
it.second.second) {
87         add(pairs[pair_idx].first, 1);
88         pair_idx++;

```

```
89         }
90         int lef = ask(it.second.first - 1);
91         int cnt = pair_idx - lef;
92         ans += cnt * it.first;
93     }
94
95     cout << ans << '\n';
96 }
97
98 signed main() {
99     ios::sync_with_stdio(0); cin.tie(0);
100     solve();
101     return 0;
102 }
```